

BÁO CÁO THÔNG TIN SẢN PHẨM, CHỨC NĂNG VÀ THÔNG SỐ KỸ THUẬT MOBILE ROBOT LIDAR

Project: Mobile Robot

Phạm vi báo cáo: nội dung dưới đây chỉ tổng hợp từ dữ liệu project, file cấu hình và tài liệu kỹ thuật đã được cung cấp. Các thông số chưa có cơ sở xác nhận được ghi rõ là chưa xác nhận hoặc cần đo/kiểm chứng thực tế.

1. Giới thiệu chung về sản phẩm

Mobile Robot LiDAR trong project này là một nền tảng robot di động tự hành được định hướng hoạt động trong môi trường trong nhà. Hệ thống sử dụng cơ cấu dẫn động vi sai (differential drive), trong đó hai bánh chủ động tạo chuyển động tịnh tiến và quay, còn một bánh tự do phía trước đóng vai trò đỡ cơ khí và ổn định hướng di chuyển.

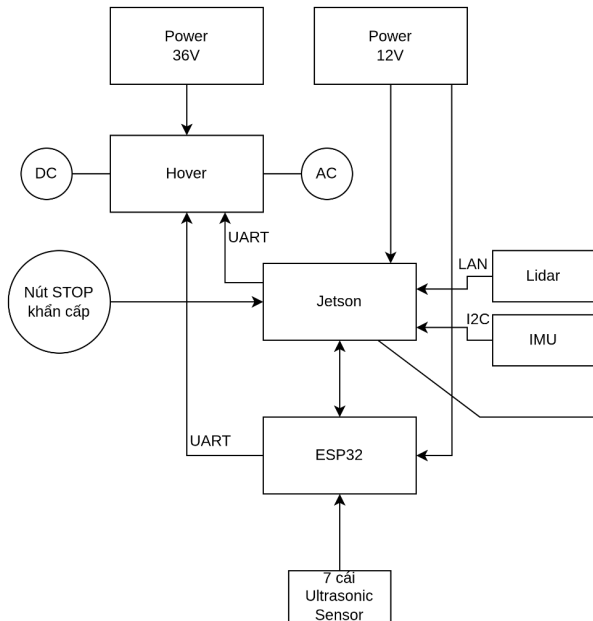
Nền tảng xử lý chính của robot là Jetson AGX Xavier Developer Kit 16GB sử dụng NVIDIA Xavier SoC, gắn trên X221-AI Carrier Board của Auvideo. Hệ thống phần mềm được xây dựng trên Ubuntu 20.04 và ROS 2 Foxy.

Về cảm biến, robot sử dụng RPLIDAR S2E kết nối qua LAN để nhận biết hình học môi trường và IMU BNO055 kết nối qua J23 của X221-AI Carrier Board để hỗ trợ ước lượng hướng/quay. Dữ liệu chuyển động bánh xe được gửi từ STM32F103RCT6 trong hoverboard về Jetson dưới dạng vận tốc góc từng bánh. Jetson dùng dữ liệu này để tính vận tốc tịnh tiến và vận tốc góc của thân robot, sau đó đưa vào bộ lọc EKF cùng với dữ liệu IMU.

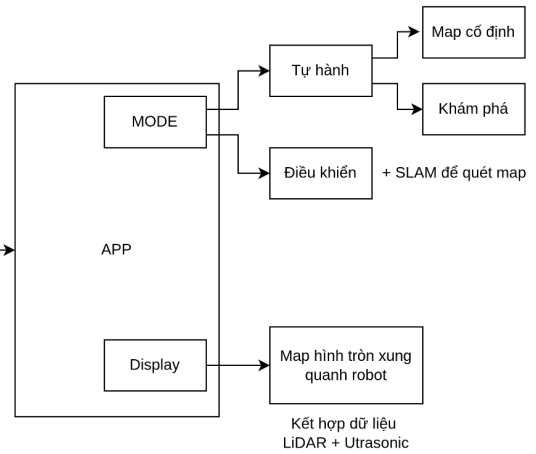
Mục tiêu của sản phẩm là phục vụ mô phỏng, định vị, lập bản đồ, điều hướng tự hành và điều hướng theo checkpoint trong môi trường trong nhà.

1.1 Sơ đồ thiết kế hệ thống

Hardware

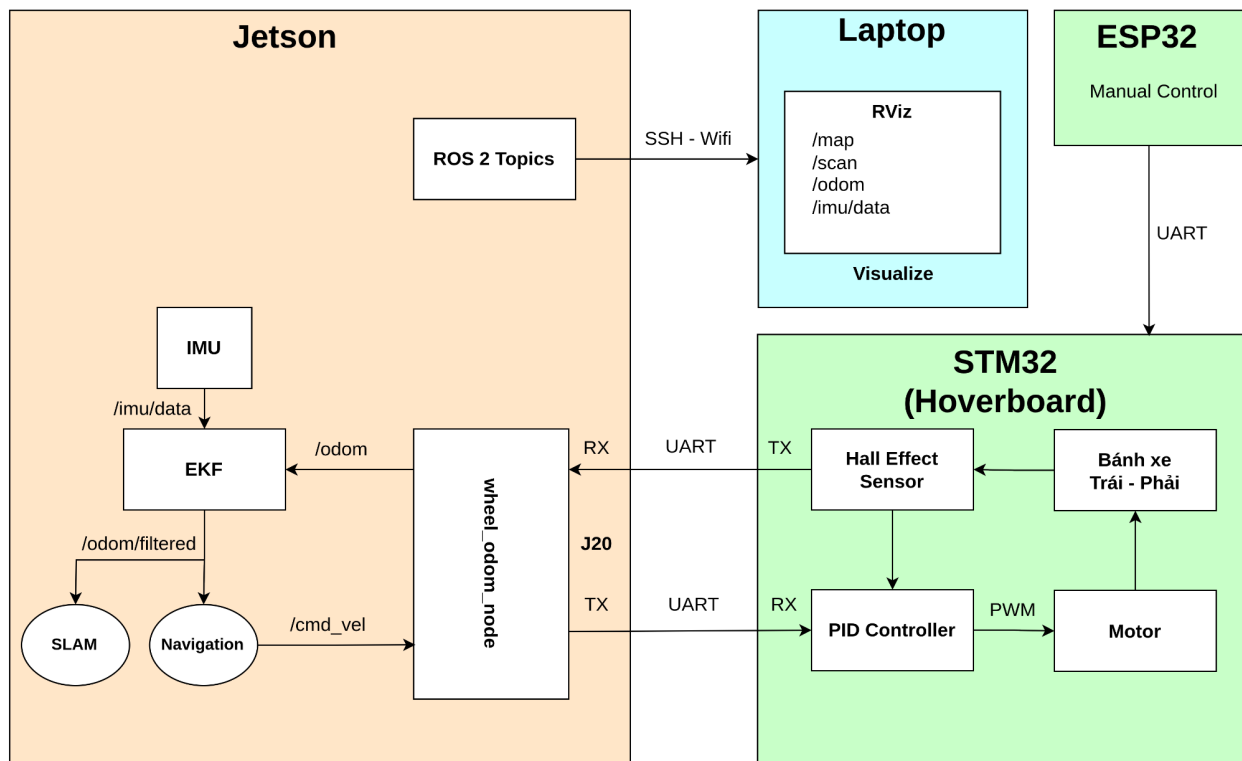


Software



Hình ảnh mang tính chất minh họa, chưa chuyên nghiệp

1.2



2. Thông tin sản phẩm và chức năng

2.1. Chức năng di chuyển và điều khiển

Robot di chuyển theo mô hình differential drive gồm hai bánh chủ động trái - phải và 1 bánh tự do.

Khả năng di chuyển: Robot có thể tiến, lùi và quay bằng cách điều khiển tương quan vận tốc giữa hai bánh. Khi hai bánh quay cùng chiều và cùng tốc độ, robot chuyển động thẳng; khi vận tốc hai bánh khác nhau, robot quay theo cung tròn; khi hai bánh quay ngược chiều, robot có thể xoay tại chỗ.

Bộ điều khiển bánh xe đặt tại STM32F103RCT6 trong hoverboard và dùng firmware hoverboard-firmware-hack-FOC. Hệ thống có hai hướng điều khiển:

- Chế độ thủ công: ESP32 gửi các lệnh như tiến, lùi hoặc rẽ đến STM32 để xử lý và điều khiển bánh xe qua giao thức UART.
- Chế độ tự hành: Jetson kết nối với STM32 thông qua module USB-to-TTL UART sử dụng cổng J33 trên Jetson board. Như vậy, Jetson đảm nhiệm vai trò xử lý cấp cao, còn STM32 thực hiện điều khiển cấp thấp liên quan trực tiếp đến bánh xe.

STM32 gửi vận tốc góc của từng bánh về Jetson. Theo mô hình động học vi sai, Jetson có thể tính vận tốc tịnh tiến v và vận tốc góc quay quanh trục z từ vận tốc góc bánh trái và bánh phải theo công thức:

$$v = r(\omega_R + \omega_L)/2$$

$$\omega_z = r(\omega_R - \omega_L)/L$$

Trong đó:

r là bán kính bánh xe.

L là khoảng cách giữa hai bánh xe.

2.2. Chức năng cảm biến và nhận thức môi trường

LiDAR RPLIDAR S2E là cảm biến chính để robot nhận biết hình học môi trường xung quanh. Dữ liệu quét LiDAR được dùng cho các thành phần như SLAM, AMCL, costmap và điều hướng. Trong cấu hình phần mềm, dữ liệu `/scan` được đưa qua chuỗi laser filter để loại bỏ các tia quét không mong muốn (ví dụ tia có thể bắn trúng thân hoặc bánh xe robot và giới hạn phạm vi đo dùng cho navigation). Sau bước lọc, hệ thống sử dụng `/scan_filtered` cho AMCL và costmap.

IMU BNO055 cung cấp dữ liệu định hướng và vận tốc góc của robot.

Dữ liệu project cũng có cấu hình phần mềm cho ultrasonic fusion. Node `ultrasonic_fusion_node_v3.py` định nghĩa tám cảm biến siêu âm theo các vị trí trên - giữa - dưới, trái - phải; xuất `/ultrasonic_scan` cho costmap, `/safety_stop` cho trạng thái dừng an toàn

và điều tiết `/cmd_vel` khi có vật cản gần. Theo node này, ngưỡng kích hoạt hard stop là 0.20 m và ngưỡng nhả hard stop là 0.30 m. Tuy nhiên, phần cứng siêu âm thực tế chưa có thông tin xác nhận đầy đủ trong dữ liệu cung cấp; vì vậy nội dung này được ghi nhận như một cấu hình phần mềm đã có, không khẳng định số lượng cảm biến thực tế đã lắp trên robot.

2.3. Chức năng định vị và hợp nhất cảm biến

Wheel odometry được tính từ vận tốc góc hai bánh do STM32 gửi về Jetson. Vì robot là dẫn động vi sai, dữ liệu đáng tin cậy nhất từ bánh xe thường là vận tốc tiến/lùi và vận tốc quay yaw. Cấu hình `ekf.yaml` thể hiện cách tiếp cận này: `/odom` chỉ bật hai thành phần `x_dot` và `yaw_dot`, trong khi không dùng trực tiếp vị trí `x`, `y` hoặc yaw từ nguồn odometry bánh xe. Cách làm này giúp EKF tự tích phân chuyển động dựa trên vận tốc và giảm phụ thuộc vào pose tức thời nếu pose đó chưa được xác nhận độc lập.

EKF trong `robot_localization` kết hợp `/odom` và `/imu/data` để tạo odometry hợp nhất. Theo cấu hình, bộ lọc chạy ở tần số 50 Hz, dùng `sensor_timeout` 0.2 s, publish TF từ `odom` đến `base_footprint` và đặt `two_d_mode = true`. Chế độ 2D phù hợp vì robot hoạt động trên mặt phẳng trong nhà; các thành phần `z`, `roll`, `pitch` và các đạo hàm không cần thiết được ép về 0 để giảm nhiễu và hạn chế sai số tích lũy.

Về mặt kỹ thuật, EKF cần thiết vì wheel odometry dễ tích lũy sai số khi bánh xe trượt, bề mặt sàn không đồng đều hoặc mô hình bán kính/khoảng cách bánh chưa khớp hoàn toàn với thực tế. IMU lại có thể bị nhiễu rung động và sai lệch theo thời gian. Khi kết hợp hai nguồn này, EKF đóng vai trò ước lượng trạng thái tối ưu hơn so với việc dùng riêng từng cảm biến, đồng thời cung cấp `/odometry/filtered` cho Nav2 và RViz.

2.4. Chức năng lập bản đồ và điều hướng tự hành

Project sử dụng `slam_toolbox` và Navigation2/Nav2 Foxy. `slam_toolbox` phục vụ chức năng lập bản đồ dựa trên dữ liệu LiDAR, còn Nav2 đảm nhiệm điều hướng tự hành trên bản đồ, bao gồm lập kế hoạch toàn cục, điều khiển cục bộ, xử lý `costmap` và hành vi phục hồi khi robot gặp vật cản hoặc bị kẹt. Trong cấu hình mô phỏng hiện tại, Nav2 bringup được khởi động với `map server`, `AMCL`, `planner server`, `controller server`, `BT navigator` và `recoveries server`.

AMCL được cấu hình với `robot_model_type = differential` và `scan_topic = /scan_filtered`. Điều này cho thấy hệ thống định vị Monte Carlo dựa trên bản đồ dùng dữ liệu LiDAR đã lọc, thay vì dùng trực tiếp `/scan` thô. BT Navigator dùng `/odometry/filtered` làm nguồn odometry phản hồi, nghĩa là Nav2 nhận trạng thái chuyển động đã qua EKF. Đây là cách tổ chức hợp lý vì bộ điều hướng không phải xử lý trực tiếp nhiễu nhiều nguồn cảm biến rời rạc.

Cấu hình `planner_server` cho thấy global planner là `nav2_navfn_planner/NavfnPlanner` với `use_astar = true`. Vì vậy, theo cấu hình hiện tại, global planning đang dùng NavFnPlanner ở chế độ A*. Local planner là `dwb_core::DWBLocalPlanner`, tức bộ điều khiển cục bộ DWB

lấy mẫu vận tốc, mô phỏng quỹ đạo ngắn hạn và chấm điểm để chọn lệnh /cmd_vel phù hợp. Báo cáo này chỉ khẳng định hệ thống được cấu hình để thực hiện các chức năng trên; chưa khẳng định robot đã chạy thành công trong thực tế nếu chưa có dữ liệu thực nghiệm xác nhận.

2.5. Chức năng điều hướng theo checkpoint

Robot có chức năng điều hướng theo checkpoint, trong đó các điểm đích được định nghĩa trong checkpoints_v2.yaml theo frame map. Mỗi checkpoint gồm ID, tên hiển thị, tọa độ x, y, z và quaternion định hướng. Node navigator.py đọc file checkpoint, nhận lệnh từ /robot/command và gửi goal đến Nav2 thông qua action NavigateToPose. Trước khi gửi goal, node còn có bước ComputePathToPose để lấy đường đi dự kiến và thực hiện pre-rotate nếu sai lệch hướng ban đầu vượt ngưỡng.

Bộ lệnh vận hành chính gồm go <id>, stop, continue và reset. Lệnh go <id> dùng để yêu cầu robot đi đến checkpoint tương ứng. Lệnh stop dừng robot tại chỗ và lưu lại mục tiêu hiện tại để có thể tiếp tục sau đó. Lệnh continue cho phép robot tính lại đường đi và tiếp tục đến mục tiêu đã lưu sau khi bị stop. Lệnh reset hủy hành trình đang dừng, chuyển robot sang trạng thái chờ; nếu không có lệnh mới trong khoảng thời gian chờ, hệ thống sẽ tự đưa robot về Home theo cấu hình home_checkpoint_id = 0. Cách tổ chức này giúp quá trình vận hành rõ ràng hơn so với việc chỉ gửi goal rời rạc trong RViz.

Bảng 1. Danh sách checkpoint theo checkpoints_v2.yaml

ID	Tên checkpoint	x (m)	y (m)	Định hướng yaw dạng quaternion	Mục đích/Ghi chú
0	Home	0.000	0.000	z=0.7085, w=0.7057	Điểm gốc/Home
1	Library	3.978	1.584	z=-0.6568, w=0.7540	Điểm đến điều hướng trong bản đồ mô phỏng
2	Meeting Room	4.003	-1.615	z=0.7097, w=0.7046	Điểm đến điều hướng trong bản đồ mô phỏng
3	Principal's Office	-4.061	1.504	z=-0.0125, w=0.9999	Điểm đến điều hướng trong bản đồ mô phỏng
4	Student Affairs	-4.067	-1.580	z=0.0228, w=0.9997	Điểm đến điều hướng trong bản đồ mô phỏng

2.6. Chức năng giám sát và vận hành

Trong vận hành thực tế, các node chính chạy trên Jetson, còn RViz chạy trên laptop để giám sát trạng thái. Launch file `rviz_launch.py` mô tả rõ cách RViz trên laptop nhận các topic từ Jetson qua mạng WiFi bằng cơ chế DDS discovery, bao gồm `/scan`, `/scan_filtered`, `/robot_description`, `/tf` và `/tf_static`. Dữ liệu project chỉ rõ middleware DDS sử dụng FastRTPS/FastDDS, tương ứng với `RMW_IMPLEMENTATION=rmw_fastrtps_cpp` nếu cấu hình môi trường yêu cầu.

Cách tách Jetson và laptop có ưu điểm thực tế. Jetson tập trung xử lý ROS 2, cảm biến, định vị và điều hướng gần robot; laptop chỉ đảm nhiệm lập trình, quan sát, debug và trực quan hóa. Điều này giảm tải hiển thị cho máy tính nhúng và phù hợp với quy trình phát triển qua Visual Studio Code Remote SSH trên Ubuntu 20.04.

3. Hướng dẫn sử dụng Mobile Robot

Khi chương trình khởi động, màn hình Main Menu sẽ hiện ra cho phép bạn chọn và điều hướng tới các tính năng của Mobile Robot bao gồm: Direction View, Chatbot, Setting.

3.1. Thanh trạng thái hệ thống (Container Bar)

Ở góc trên cùng của màn hình, bạn luôn có thể theo dõi các thông số quan trọng của hệ thống trong thời gian thực, bao gồm:

- **Thời gian hệ thống:** Đồng bộ chính xác với thời gian thực.
- **Trạng thái Wi-Fi:** Hiển thị biểu tượng kết nối mạng và cường độ tín hiệu.

3.2. Điều hướng và Chọn điểm đến (Direction View)

Đây là nơi bạn chỉ định vị trí mà robot cần đi tới:

- Bấm vào nút **Bản đồ (Map)** để hiển thị danh sách các phòng hoặc các điểm đến đã được lưu sẵn.
- Chạm vào một **địa điểm (Location)** trên bản đồ hoặc danh sách từ nút bản đồ mà bạn muốn robot di chuyển tới.
- Một **hộp thoại xác nhận (Confirmation Popup)** sẽ hiện ra để tránh việc chạm nhầm. Sau khi bạn nhấn "Đồng ý", robot sẽ bắt đầu lộ trình của mình.

3.3. Theo dõi quá trình di chuyển (Running View)

Khi robot bắt đầu di chuyển, giao diện sẽ tự động chuyển sang màn hình theo dõi hành trình:

- **Trạng thái:** Bạn sẽ thấy trạng thái của robot hiển thị rõ ràng là đang di chuyển (*Navigating*) hay đang dừng lại (*Stopped*).

- **Điều khiển tức thời:** Trong quá trình robot chạy, bạn có thể can thiệp bằng các nút bấm:
 - **Stop:** Tạm dừng robot ngay lập tức.
 - **Continue:** Ra lệnh cho robot tiếp tục hành trình đang dang dở.
 - **Reset:** Hủy bỏ lộ trình hiện tại và cho phép chọn địa điểm mới. Robot sẽ tự động quay trở về "Trạm sạc / Vị trí xuất phát" (Direction View) sau 30 giây đếm ngược.

3.4. Trợ lý ảo thông minh (Chatbot):

Ứng dụng được tích hợp sẵn một Chatbot thông minh giúp tăng cường trải nghiệm tương tác với robot:

- **Hỗ trợ trực tiếp:** Bạn có thể trò chuyện với Chatbot ngay trên màn hình để đặt câu hỏi về cách sử dụng, hỏi trạng thái của robot, hoặc tra cứu các thông tin liên quan.
- **Thao tác dễ dàng:** Mở giao diện Chatbot và nhập nội dung câu hỏi thông qua bàn phím ảo hiển thị trên màn hình.
- **Phản hồi tức thì:** Chatbot sẽ phân tích yêu cầu của bạn và đưa ra câu trả lời hoặc hướng dẫn xử lý một cách nhanh chóng và chính xác.

4. Thông số kỹ thuật sản phẩm

Các thông số dưới đây được phân nhóm theo nền tảng tính toán, cảm biến, cơ cấu di chuyển, phần mềm ROS 2, định vị - điều hướng, bản đồ và checkpoint. Những giá trị lấy từ file cấu hình được hiểu là thông số cấu hình hiện tại; nếu cần công bố như thông số sản phẩm thực tế thì phải đo và kiểm chứng trên robot thật.

4.1. Thông số nền tảng tính toán

Bảng 2. Thông số nền tảng tính toán

Hạng mục	Thông số	Ghi chú
Máy tính nhúng chính	Jetson AGX Xavier Developer Kit 16GB	Theo dữ liệu project; dùng NVIDIA Xavier SoC.
Carrier board	Auvideo X221-AI	Tài liệu X221 manual được dùng để đối chiếu cổng J23/J33.
Hệ điều hành trên Jetson	Ubuntu 20.04	Theo dữ liệu project.
ROS	ROS 2 Foxy	Theo dữ liệu project và package.xml.
Middleware DDS	FastRTPS/FastDDS	Theo dữ liệu project; laptop và Jetson cần cùng cấu hình DDS/domain khi chạy qua WiFi.

Máy phát triển	Laptop Ubuntu 20.04, Visual Studio Code Remote SSH	Kết nối Jetson qua SSH bằng WiFi.
----------------	--	-----------------------------------

4.2. Thông số cảm biến

Bảng 3. Thông số cảm biến và lọc dữ liệu

Hạng mục	Thông số	Ghi chú
LiDAR	RPLIDAR S2E, kết nối LAN	Dùng cho /scan, SLAM, AMCL, costmap và navigation.
IMU	BNO055, kết nối I2C qua J23 của X221-AI	Theo dữ liệu project và bno055_params.yaml.
BNO055 topic prefix	imu/	Dữ liệu IMU được tổ chức dưới namespace imu/.
BNO055 frame_id	imu_link	Frame quan trọng trong TF và EKF.
BNO055 I2C bus	/dev/i2c-8	Theo bno055_params.yaml; ghi chú cấu hình dùng I2C_GP5 trên J23.
BNO055 I2C address	0x28	Theo bno055_params.yaml.
BNO055 data query frequency	50 Hz	Tần số truy vấn dữ liệu trong cấu hình driver.
LiDAR filter - range filter	lower=0.06 m; upper=15.0 m	Giới hạn phạm vi sử dụng cho dữ liệu scan đã lọc.
LiDAR filter - box filter	x=[-0.13, 0.33], y=[-0.31, 0.31], z=[-0.05, 1.4]	Vùng lọc tia có khả năng trùng thân/bánh xe robot.
Ultrasonic fusion	8 topic cảm biến trong node; hard stop ON < 0.20 m, OFF > 0.30 m	Cấu hình phần mềm có node ultrasonic fusion; phần cứng thực tế cần xác nhận.

4.3. Thông số cơ cấu di chuyển

Bảng 4. Thông số cơ cấu di chuyển và wheel odometry

Hạng mục	Thông số	Ghi chú
Kiểu dẫn động	Differential drive	Hai bánh chủ động trái/phải và một bánh tự do phía trước.
Bộ điều khiển bánh xe	STM32F103RCT6 trong hoverboard	Điều khiển cấp thấp cho bánh xe.
Firmware STM32	hoverboard-firmware-hack-FOC	Theo dữ liệu project.
Điều khiển thủ công	ESP32 -> UART -> STM32	ESP32 gửi lệnh thủ công cho STM32.

Điều khiển tự hành	Jetson -> USB-to-TTL UART -> STM32, dùng J33 trên Jetson board	Jetson gửi lệnh điều hướng/điều khiển cấp cao.
Dữ liệu phản hồi từ STM32	Vận tốc góc từng bánh xe	STM32 không gửi tick encoder trái/phải theo dữ liệu project.
Serial wheel odometry	/dev/ttyUSB0, 115200 baud	Theo wheel_odom_params.yaml.
Bán kính bánh xe	0.08255 m	Theo cấu hình wheel odometry; cần đo/kiểm chứng thực tế nếu dùng làm thông số cơ khí chính thức.
Khoảng cách hai bánh / wheel base	0.42109 m	Theo cấu hình wheel odometry.
publish_tf của wheel odom	false	EKF publish TF, node wheel odom không publish TF.

4.4. Thông số phần mềm ROS 2

Bảng 5. Thông số phần mềm ROS 2

Hạng mục	Thông số	Ghi chú
Package ROS 2	mobile_robot	Theo package.xml và CMakeLists.txt.
Nhóm chức năng package	SLAM, Navigation2, Gazebo Simulation, EKF, checkpoint navigator, laser filter, ultrasonic fusion	Theo package.xml, CMakeLists.txt và launch files.
robot_state_publisher	Có trong launch mô phỏng	Publish robot_description và TF tĩnh/động từ URDF.
EKF node	robot_localization/ekf_node	Dùng ekf.yaml.
Laser filter	laser_filters/scan_to_scan_filte r_chain	Lọc /scan -> /scan_filtered.
Nav2	nav2_bringup/bringup_launch. py	Dùng nav2_params_v2.yaml và map file.
Checkpoint navigator	navigator.py	Nhận /robot/command, publish /robot/state, /robot/current_checkpoint, /robot/status_message.
Ultrasonic fusion	ultrasonic_fusion_node_v3.py	Xuất /ultrasonic_scan, /safety_stop và gate /cmd_vel.
Các topic chính	/odom, /odometry/filtered, /imu/data, /scan, /scan_filtered, /cmd_vel, /cmd_vel_nav, /robot/command, /robot/state	Tổng hợp từ ekf.yaml, nav2_params_v2.yaml, launch và các node Python.
Các frame chính	map, odom, base_footprint, imu_link, laser_frame	Tổng hợp từ EKF, Nav2, BNO055 và URDF.

4.5. Thông số định vị và điều hướng

Bảng 6. Thông số định vị và điều hướng

Hạng mục	Thông số	Ghi chú
EKF frequency	50.0 Hz	Theo ekf.yaml.
EKF two_d_mode	true	Phù hợp robot chạy trên mặt phẳng trong nhà.
EKF input odom0	/odom	Dùng x_dot và yaw_dot từ wheel odometry.
EKF input imu0	/imu/data	Dùng yaw và yaw_dot từ IMU theo cấu hình.
Odometry cho Nav2	/odometry/filtered	BT Navigator dùng odometry đã hợp nhất.
AMCL scan topic	/scan_filtered	AMCL dùng scan đã lọc.
AMCL robot model	differential	Mô hình chuyển động differential.
AMCL laser range	min=0.07 m; max=14.9 m	Phạm vi hiệu dụng cấp cho AMCL.
AMCL particles	min=500; max=2000	Số hạt của bộ lọc Monte Carlo.
Controller frequency	20.0 Hz	Tần số xuất lệnh điều khiển cục bộ.
Local planner	dwb_core::DWBLocalPlanner	DWBLocalPlanner.
Global planner	nav2_navfn_planner/NavfnPlanner	NavFnPlanner.
use_astar	true	Global planning dùng NavFnPlanner ở chế độ A*.
Planner tolerance	0.25 m	Dung sai tìm đường đến đích.
Goal tolerance	xy=0.25 m; yaw=0.2 rad	Theo SimpleGoalChecker.
Giới hạn vận tốc phần mềm	max_vel_x=0.4 m/s; max_vel_theta=1.5 rad/s	Là giới hạn Nav2, không phải tốc độ tối đa thực nghiệm của robot.
Giới hạn gia tốc phần mềm	acc_lim_x=0.1 m/s ² ; acc_lim_theta=1.0 rad/s ²	Theo DWB planner.
Giới hạn giảm tốc phần mềm	decel_lim_x=-0.35 m/s ² ; decel_lim_theta=-1.0 rad/s ²	Theo DWB planner.
Recovery behaviors	wait, spin, backup	wait, spin, backup.

4.6. Thông số costmap, bản đồ và checkpoint

Bảng 7. Thông số costmap, bản đồ và checkpoint

Hạng mục	Thông số	Ghi chú
----------	----------	---------

Map sim_map.yaml	image=sim_map.pgm; resolution=0.05 m/pixel; origin=[-14.9, -2.4, 0]	Bản đồ dùng trong sim_nav_v2.launch.py.
Map map_test.yaml	image=map_test.pgm; resolution=0.05 m/pixel; origin=[-0.1, -0.1, 0.0]	Bản đồ kiểm thử corridor test map.
Local costmap frame	odom	Costmap cục bộ bám theo odom.
Local costmap size/resolution	width=5 m; height=5 m; resolution=0.03 m/cell	Rolling window quanh robot.
Local costmap footprint	[[0.32, 0.30], [0.32, -0.30], [-0.12, -0.30], [-0.12, 0.30]]	Footprint dùng cho kiểm tra va chạm; không xem là kích thước tổng thể robot nếu chưa đo thực tế.
Local costmap obstacle range	obstacle_max_range=10.0 m; raytrace_max_range=10.5 m	Dùng /scan_filtered.
Local inflation	inflation_radius=0.12 m; cost_scaling_factor=1.0	Vùng phòng vật cản cục bộ.
Global costmap frame	map	Costmap toàn cục neo vào map.
Global costmap resolution	0.03 m/cell	Theo nav2_params_v2.yaml.
Global costmap obstacle range	obstacle_max_range=14.0 m; raytrace_max_range=14.5 m	Dùng /scan_filtered.
Global inflation	inflation_radius=0.25 m; cost_scaling_factor=1.5	Vùng phòng vật cản toàn cục.
Checkpoint frame	map	Các checkpoint được định nghĩa trong frame map.

4.7. Các thông số chưa xác nhận hoặc cần kiểm chứng thực tế

Một số thông số thường gặp trong tài liệu sản phẩm chưa có dữ liệu xác nhận đáng tin cậy hoặc mới chỉ xuất hiện như tham số mô phỏng/cấu hình. Vì vậy, các thông số dưới đây không được kết luận như đặc tính sản phẩm thực tế trong báo cáo này.

Bảng 8. Thông số chưa xác nhận

Thông số	Trạng thái	Ghi chú
Khối lượng tổng thể robot	Chưa có thông tin xác nhận trong dữ liệu cung cấp	URDF có tham số mô phỏng nhưng chưa được dùng như giá trị đo thực tế.
Kích thước tổng thể robot	Chưa có thông tin xác nhận trong dữ	Footprint Nav2 chỉ phục vụ kiểm tra va chạm trong costmap.
Dung lượng pin / thời gian hoạt động	Chưa có thông tin xác nhận trong dữ liệu cung cấp	Sơ đồ kiến trúc có nhận nguồn nhưng chưa đủ để kết luận pin.

Tải trọng	Chưa có thông tin xác nhận trong dữ liệu cung cấp	Cần đo/kiểm chứng thực tế.
Tốc độ tối đa thực nghiệm	60rpm	Nav2 chỉ cung cấp giới hạn vận tốc phần mềm.
Sai số định vị / sai số đến checkpoint	Cần đo/kiểm chứng thực tế	Cần log thực nghiệm hoặc đánh giá trên bản đồ thật.
Số lượng phần cứng siêu âm thực tế	Chưa có thông tin xác nhận thống nhất	Sơ đồ kiến trúc có đề cập nhóm cảm biến siêu âm; node phần mềm hiện tại định nghĩa 8 cảm biến.

5. Nhận xét kỹ thuật

Kiến trúc Jetson + STM32 là một lựa chọn phù hợp cho robot di động tự hành. Jetson AGX Xavier có vai trò xử lý cấp cao: chạy ROS 2, nhận dữ liệu cảm biến, lọc LiDAR, thực hiện EKF, chạy AMCL, Nav2 và các node điều hướng checkpoint. Trong khi đó, STM32F103RCT6 trong hoverboard xử lý điều khiển bánh xe ở tầng thấp hơn. Cách phân tầng này tránh việc Jetson phải trực tiếp xử lý các vòng điều khiển thời gian thực của động cơ, đồng thời cho phép ROS 2 tập trung vào bài toán nhận thức, định vị và lập kế hoạch.

Việc kết hợp LiDAR, IMU và wheel odometry cũng có cơ sở kỹ thuật rõ ràng. LiDAR cung cấp thông tin hình học môi trường, giúp robot khớp vị trí trên bản đồ thông qua AMCL hoặc tạo bản đồ thông qua SLAM. Wheel odometry cung cấp chuyển động tương đối liên tục, đặc biệt hữu ích trong khoảng thời gian ngắn giữa các lần cập nhật cảm biến. IMU hỗ trợ đo hướng và vận tốc góc, giúp giảm sai lệch yaw do trượt bánh hoặc sai số tích lũy. Khi đưa các nguồn dữ liệu này vào EKF, hệ thống có thể tạo ra trạng thái odometry ổn định hơn để Nav2 sử dụng.

ROS 2 Foxy và Nav2 phù hợp với bài toán robot trong nhà vì cung cấp đầy đủ các thành phần cần thiết: TF, robot_state_publisher, lifecycle node, map server, AMCL, planner server, controller server, behavior tree navigator, recovery behaviors và RViz. Đặc biệt, cấu hình hiện tại đã tách rõ vai trò của global planner NavFnPlanner ở chế độ A* và local planner DWBLocalPlanner. Điều này giúp robot có thể lập đường đi trên bản đồ toàn cục và đồng thời phản ứng cục bộ với vật cản trong costmap.

Tuy nhiên, một số thông số vẫn cần được kiểm chứng nếu chuyển từ mô phỏng hoặc cấu hình sang sản phẩm thực tế. Các giới hạn vận tốc/gia tốc trong Nav2 là giới hạn phần mềm; chúng không thay thế cho tốc độ tối đa đo trên robot thật. Tương tự, footprint trong costmap không nên được xem là kích thước tổng thể robot nếu chưa đo cơ khí. Các thông số như khối lượng, pin, thời gian hoạt động, tải trọng, sai số định vị và độ tin cậy khi chạy lâu dài cần được đo bằng thực nghiệm.

6. Kết luận ngắn

Mobile Robot LiDAR trong project là một nền tảng robot di động trong nhà được xây dựng trên ROS 2 Foxy và Ubuntu 20.04. Hệ thống có khả năng mô phỏng, định vị, lập bản đồ, điều hướng tự hành và điều hướng theo checkpoint. Kiến trúc tổng thể được tổ chức theo các lớp tương đối rõ: lớp cảm biến gồm LiDAR, IMU và cấu hình siêu âm; lớp xử lý nhúng dùng Jetson AGX Xavier; lớp điều khiển bánh xe dùng STM32 trong hoverboard; lớp phần mềm robot dùng ROS 2/Nav2/slam_toolbox; và lớp giám sát dùng RViz trên laptop qua mạng WiFi.

Theo cấu hình hiện tại, robot sử dụng EKF để hợp nhất wheel odometry và IMU, AMCL để định vị trên bản đồ, NavFnPlanner ở chế độ A* cho lập kế hoạch toàn cục và DWBLocalPlanner cho điều khiển cục bộ. Các checkpoint Home, Library, Meeting Room, Principal's Office và Student Affairs đã được định nghĩa trong frame map, cho phép triển khai kịch bản robot đi đến các vị trí mục tiêu trong môi trường mô phỏng hoặc bản đồ trong nhà. Những thông số chưa được đo hoặc chưa có xác nhận thực tế đã được tách riêng để tránh nhầm lẫn giữa cấu hình phần mềm và đặc tính sản phẩm thật.

7. Nguồn dữ liệu sử dụng

Bảng 9. Nguồn dữ liệu được sử dụng trong báo cáo

Nguồn	Nội dung đã sử dụng
Dữ liệu project do người dùng cung cấp	Thông tin phần cứng, hệ điều hành, ROS 2, DDS, cảm biến, cơ cấu điều khiển và chế độ vận hành.
SystemArchitecture_MobileRobot.pdf	Đối chiếu kiến trúc phần cứng/phần mềm, data flow, control flow và vai trò Jetson - laptop - ESP32 - STM32.
X221_Manual_v2.0.pdf	Đối chiếu thông tin carrier board X221-AI, đặc biệt các cổng J23 và J33.
package.xml	Xác nhận package mobile_robot, dependencies ROS 2 và mô tả hỗ trợ SLAM, Navigation2, Gazebo Simulation.
CMakeLists.txt	Xác nhận các thư mục, launch/config/world/scripts được install và các Python executable.
ekf.yaml	Thông số EKF, input /odom, /imu/data, frame map/odom/base_footprint và two_d_mode.
nav2_params_v2.yaml	Thông số AMCL, BT Navigator, DWBLocalPlanner, NavFnPlanner/A*, costmap và recovery behaviors.
wheel_odom_params.yaml	Thông số serial, wheel_radius, wheel_base, odom_frame và base_frame cho wheel odometry.

bno055_params.yaml	Thông số BNO055: I2C, bus, address, frame_id, topic prefix và frequency.
checkpoints_v2.yaml	Danh sách checkpoint, tọa độ và định hướng trong frame map.
laser_filter.yaml	Thông số lọc LiDAR: box filter, range filter và angular bounds filter.
sim_map.yaml, map_test.yaml	Thông số bản đồ: image, resolution, origin, occupied/free thresholds.
sim_nav_v2.launch.py, spawn_sim_robot_v2.launch.py, rviz_launch.py	Cấu trúc khởi chạy mô phỏng, Nav2, RViz, EKF, laser filter, ultrasonic fusion và checkpoint navigator.
mobile_robot_v3.urdf.xacro, gazebo_control_v3.xacro	Mô hình robot, frame base_footprint/laser_frame/imu_link, plugin diff drive, LiDAR, IMU và cảm biến siêu âm trong Gazebo.
navigator.py, checkpoint_cmd_v2.py, ultrasonic_fusion_node_v3.py, imu_reader.py	Logic điều hướng checkpoint, giao diện lệnh, hard stop siêu âm và đọc/chuyển đổi dữ liệu IMU.